

Notes on Chapter 25 - Functions

Norman Lim

2025-08-15

Prerequisites

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.0.4
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(nycflights13)
```

Vector functions

Vector functions are functions that take one or more vectors and return a vector result.

Example:

```
df <- tibble(
  a = rnorm(5),
  b = rnorm(5),
  c = rnorm(5),
  d = rnorm(5),
)

df |> mutate(
  a = (a - min(a, na.rm = TRUE)) / (max(a, na.rm = TRUE) - min(a, na.rm = TRUE)),
  b = (b - min(b, na.rm = TRUE)) / (max(b, na.rm = TRUE) - min(b, na.rm = TRUE)),
  c = (c - min(c, na.rm = TRUE)) / (max(c, na.rm = TRUE) - min(c, na.rm = TRUE)),
  d = (d - min(d, na.rm = TRUE)) / (max(d, na.rm = TRUE) - min(d, na.rm = TRUE))
)

## # A tibble: 5 x 4
##       a         b         c         d
```

```
##    <dbl>  <dbl> <dbl> <dbl>
## 1 0      -0.106 1      0.687
## 2 0.589  0.126 0      1
## 3 1      0.216 0.417 0
## 4 0.388 -0.784 0.205 0.411
## 5 0.501 -0.106 0.740 0.748
```

There is a mistake in `b`. This could have been avoided if we used functions.

Three things needed to make a function:

- A **name**.
- The **arguments**.
- The **body**.

Format:

```
name <- function(x) {
  body
}
```

Creating the function for the rescaling code above:

```
rescale01 <- function(x) {
  (x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
}
```

Testing if the function works as intended:

```
rescale01(c(-10, 0, 10))
```

```
## [1] 0.0 0.5 1.0
```

```
rescale01(c(1, 2, 3, NA, 5))
```

```
## [1] 0.00 0.25 0.50 NA 1.00
```

Using the function (or calling the function):

```
df |> mutate(
  a = rescale01(a),
  b = rescale01(b),
  c = rescale01(c),
  d = rescale01(d)
)
```

```
## # A tibble: 5 x 4
##       a      b      c      d
##   <dbl> <dbl> <dbl> <dbl>
## 1 0      0.679 1      0.687
## 2 0.589 0.910 0      1
## 3 1      1      0.417 0
## 4 0.388 0      0.205 0.411
## 5 0.501 0.678 0.740 0.748
```

Improving the function

Replacing the repetitious parts of the function:

```
rescale01 <- function(x) {  
  rng <- range(x, na.rm = TRUE)  
  (x - rng[1]) / (rng[2] - rng[1])  
}
```

Retesting:

```
x <- c(1:10, Inf)  
rescale01(x)
```

```
## [1] 0 0 0 0 0 0 0 0 0 0 NaN
```

The result is not particularly useful. It is better to ignore infinite values:

```
rescale01 <- function(x) {  
  rng <- range(x, na.rm = TRUE, finite = TRUE)  
  (x - rng[1]) / (rng[2] - rng[1])  
}
```

Retesting:

```
rescale01(x)
```

```
## [1] 0.0000000 0.1111111 0.2222222 0.3333333 0.4444444 0.5555556 0.6666667  
## [8] 0.7777778 0.8888889 1.0000000          Inf
```

Mutate functions

A simple variation of `rescale01`:

```
z_score <- function(x) {  
  (x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE)  
}
```

Another variation using `case_when()`:

```
clamp <- function(x, min, max) {  
  case_when(  
    x < min ~ min,  
    x > max ~ max,  
    .default = x  
  )  
}  
  
clamp(1:10, min = 3, max = 7)
```

```
## [1] 3 3 3 4 5 6 7 7 7 7
```

A function that makes the first character upper case:

```
first_upper <- function(x) {  
  str_sub(x, 1, 1) <- str_to_upper(str_sub(x, 1, 1))  
  x  
}  
  
first_upper("hello")
```

```
## [1] "Hello"
```

A function for stripping percent signs:

```
# https://twitter.com/NVlabormarket/status/1571939851922198530  
clean_number <- function(x) {  
  is_pct <- str_detect(x, "%")  
  num <- x |>  
    str_remove_all("%") |>  
    str_remove_all(",") |>  
    str_remove_all(fixed("$")) |>  
    as.numeric()  
  if_else(is_pct, num / 100, num)  
}  
  
clean_number("$12,300")
```

```
## [1] 12300
```

```
clean_number("45%")
```

```
## [1] 0.45
```

A function for replacing 997, 998, or 999 with NA:

```
fix_na <- function(x) {  
  if_else(x %in% c(997, 998, 999), NA, x)  
}
```

Summary functions

These are functions that return a single value for use in `summarize()`.

Example:

```
commas <- function(x) {  
  str_flatten(x, collapse = ", ", last = " and ")  
}  
  
commas(c("cat", "dog", "pigeon"))
```

```
## [1] "cat, dog and pigeon"
```

Another example with mathematical computation – coefficient of variation:

```
cv <- function(x, na.rm = FALSE) {  
  sd(x, na.rm = na.rm) / mean(x, na.rm = na.rm)  
}
```

```
cv(runif(100, min = 0, max = 50))
```

```
## [1] 0.5819239
```

```
cv(runif(100, min = 0, max = 500))
```

```
## [1] 0.6577771
```

An example for making handling missing data more “readable”:

```
n_missing <- function(x) {  
  sum(is.na(x))  
}
```

Another example with multiple vector inputs:

```
mape <- function(actual, predicted) {  
  sum(abs((actual - predicted) / actual)) / length(actual)  
}
```

Data frame functions

Embracing (wrapping a variable in double braces) to tell dplyr to use the value stored inside the argument:

```
grouped_mean <- function(df, group_var, mean_var) {  
  df |>  
    group_by({{ group_var }}) |>  
    summarize(mean({{ mean_var }}))  
}
```

```
df <- tibble(  
  mean_var = 1,  
  group_var = "g",  
  group = 1,  
  x = 10,  
  y = 100  
)
```

```
df |> grouped_mean(group, x)
```

```
## # A tibble: 1 x 2  
##   group 'mean(x)'  
##   <dbl>   <dbl>  
## 1     1       10
```

Without embracing, the function above will throw an error. This problem happens because dplyr uses **tidy evaluation** to allow referring to the names of variables inside the data frame without any special treatment.

When to use embracing?

- **Data-masking:** used in functions like `arrange()`, `filter()`, and `summarize()` that compute with variables.
- **Tidy-selection:** used for functions like `select()`, `relocate()`, and `rename()` that select variables.

Common use cases of embracing

```
summary6 <- function(data, var) {  
  data |> summarize(  
    min = min({{ var }}, na.rm = TRUE),  
    mean = mean({{ var }}, na.rm = TRUE),  
    median = median({{ var }}, na.rm = TRUE),  
    max = max({{ var }}, na.rm = TRUE),  
    n = n(),  
    n_miss = sum(is.na({{ var }})),  
    .groups = "drop"  
  )  
}  
  
diamonds |> summary6(carat)
```

```
## # A tibble: 1 x 6  
##   min mean median max      n n_miss  
##   <dbl> <dbl> <dbl> <dbl> <int> <int>  
## 1  0.2 0.798    0.7  5.01 53940     0
```

Tip: Whenever you wrap `summarize()` in a helper, it is a good idea to set `.groups = "drop"` to both avoid seeing the warning and leaving the data in an ungrouped state.

Example using the function above on grouped data:

```
diamonds |>  
  group_by(cut) |>  
  summary6(carat)
```

```
## # A tibble: 5 x 7  
##   cut      min mean median max      n n_miss  
##   <ord>   <dbl> <dbl> <dbl> <dbl> <int> <int>  
## 1 Fair    0.22 1.05    1    5.01 1610     0  
## 2 Good    0.23 0.849  0.82  3.01 4906     0  
## 3 Very Good 0.2  0.806  0.71  4    12082    0  
## 4 Premium 0.2  0.892  0.86  4.01 13791    0  
## 5 Ideal  0.2  0.703  0.54  3.5 21551    0
```

Example using the function above on computed variables:

```
diamonds |>  
  group_by(cut) |>  
  summary6(log10(carat))
```

```
## # A tibble: 5 x 7
##   cut      min    mean  median    max      n n_miss
##   <ord>    <dbl>  <dbl>   <dbl> <dbl> <int>  <int>
## 1 Fair     -0.658 -0.0273  0      0.700  1610    0
## 2 Good     -0.638 -0.133  -0.0862 0.479  4906    0
## 3 Very Good -0.699 -0.164  -0.149  0.602 12082    0
## 4 Premium  -0.699 -0.125  -0.0655 0.603 13791    0
## 5 Ideal    -0.699 -0.225  -0.268  0.544 21551    0
```

Example using the helper function `count()` to compute proportions:

```
count_prop <- function(df, var, sort = FALSE) {
  df |>
    count({{ var }}, sort = sort) |>
    mutate(prop = n / sum(n))
}

diamonds |> count_prop(clarity)
```

```
## # A tibble: 8 x 3
##   clarity      n    prop
##   <ord>    <int>  <dbl>
## 1 I1         741 0.0137
## 2 SI2        9194 0.170
## 3 SI1       13065 0.242
## 4 VS2       12258 0.227
## 5 VS1        8171 0.151
## 6 VVS2        5066 0.0939
## 7 VVS1        3655 0.0678
## 8 IF         1790 0.0332
```

Example with conditional variables as input argument:

```
unique_where <- function(df, condition, var) {
  df |>
    filter({{ condition }}) |>
    distinct({{ var }}) |>
    arrange({{ var }})
}

# Find all the destinations in December
flights |> unique_where(month == 12, dest)
```

```
## # A tibble: 96 x 1
##   dest
##   <chr>
## 1 ABQ
## 2 ALB
## 3 ATL
## 4 AUS
## 5 AVL
## 6 BDL
```

```
## 7 BGR
## 8 BHM
## 9 BNA
## 10 BOS
## # i 86 more rows
```

Example of a function that automatically selects a predefined set of columns (aka features):

```
subset_flights <- function(rows, cols) {
  flights |>
    filter({{ rows }}) |>
    select(time_hour, carrier, flight, {{ cols }})
}
```

Data-masking vs. tidy-selection

Using the `pick()` function to make tidy-selection work inside a data-masking function:

```
count_missing <- function(df, group_vars, x_var) {
  df |>
    group_by(pick({{ group_vars }})) |>
    summarize(
      n_miss = sum(is.na({{ x_var }})),
      .groups = "drop"
    )
}

flights |>
  count_missing(c(year, month, day), dep_time)
```

```
## # A tibble: 365 x 4
##   year month   day n_miss
##   <int> <int> <int>   <int>
## 1  2013     1     1     4
## 2  2013     1     2     8
## 3  2013     1     3    10
## 4  2013     1     4     6
## 5  2013     1     5     3
## 6  2013     1     6     1
## 7  2013     1     7     3
## 8  2013     1     8     4
## 9  2013     1     9     5
## 10 2013     1    10     3
## # i 355 more rows
```

Example using `pick()` to make 2d table of columns:

```
count_wide <- function(data, rows, cols) {
  data |>
    count(pick({{ rows }}, {{ cols }})) |>
    pivot_wider(
      names_from = {{ cols }},
```

```

    values_from = n,
    names_sort = TRUE,
    values_fill = 0
  )
}

diamonds |> count_wide(c(clarity, color), cut)

```

```

## # A tibble: 56 x 7
##   clarity color Fair Good 'Very Good' Premium Ideal
##   <ord>   <ord> <int> <int>      <int>      <int> <int>
## 1 I1      D      4      8          5         12     13
## 2 I1      E      9     23         22         30     18
## 3 I1      F     35     19         13         34     42
## 4 I1      G     53     19         16         46     16
## 5 I1      H     52     14         12         46     38
## 6 I1      I     34      9          8         24     17
## 7 I1      J     23      4          8         13      2
## 8 SI2     D     56    223        314        421    356
## 9 SI2     E     78    202        445        519    469
## 10 SI2    F     89    201        343        523    453
## # i 46 more rows

```

Exercises

- Using the datasets from 'nycflights13', write a function that:
 - Finds all flights that were cancelled.
 - Counts the number of cancelled flights and the number of flights delayed by more than an hour.
 - Finds all flights that were cancelled or delayed by more than a user-supplied number of hours.
 - Summarizes the weather to compute the minimum, mean, and maximum, of a user-supplied variable:
 - Converts the user-supplied variable that uses clock time (e.g. `dep_time`, `arr_time`, etc.) into a decimal time (i.e. `hours + (minutes/60)`).
- For each of the following functions, list all arguments that use tidy evaluation and describe whether they use data-masking or tidy-selection: `distinct()`, `count()`, `group_by()`, `rename_width()`, `slice_min()`, `slice_sample()`.
- Generalize the following function so that you can supply any number of variables to count.

```

count_prop <- function(df, var, sort = FALSE) {
  df |>
    count({{ var }}, sort = sort) |>
    mutate(prop = n / sum(n))
}

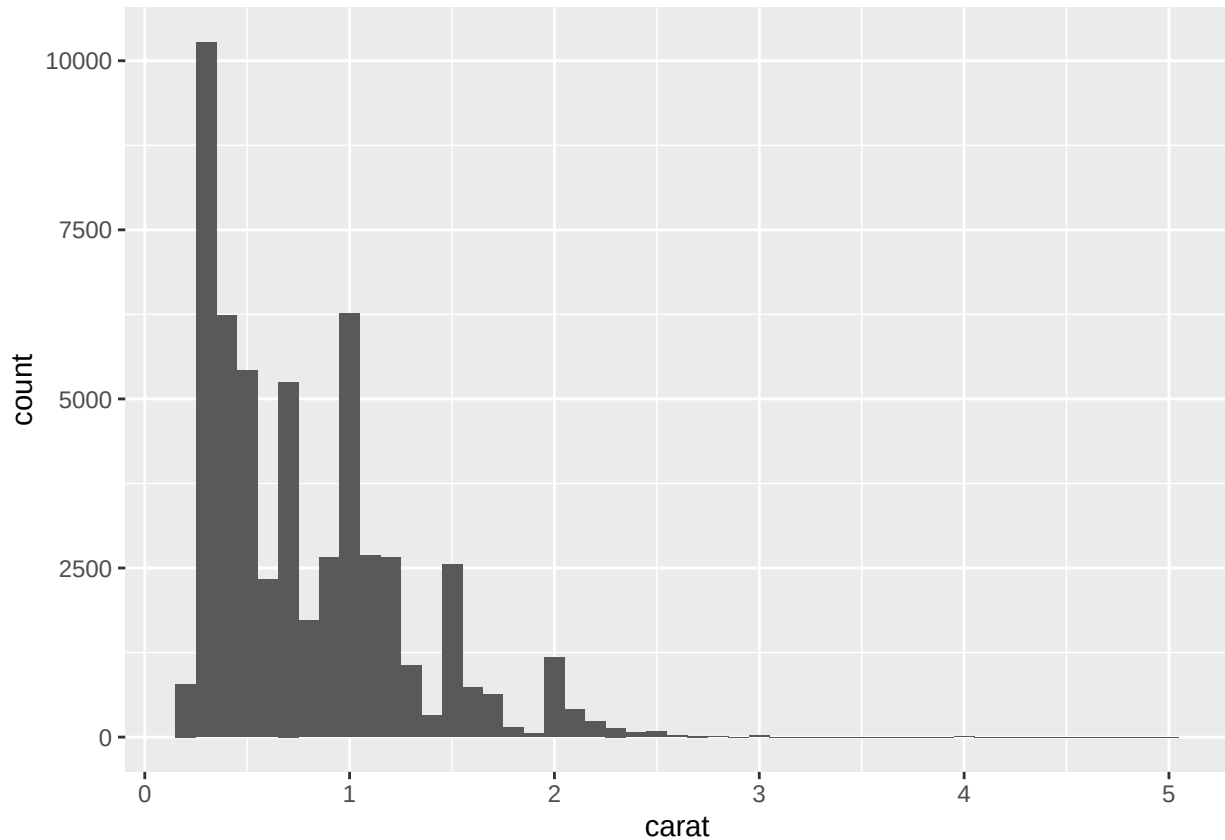
```

Plot functions

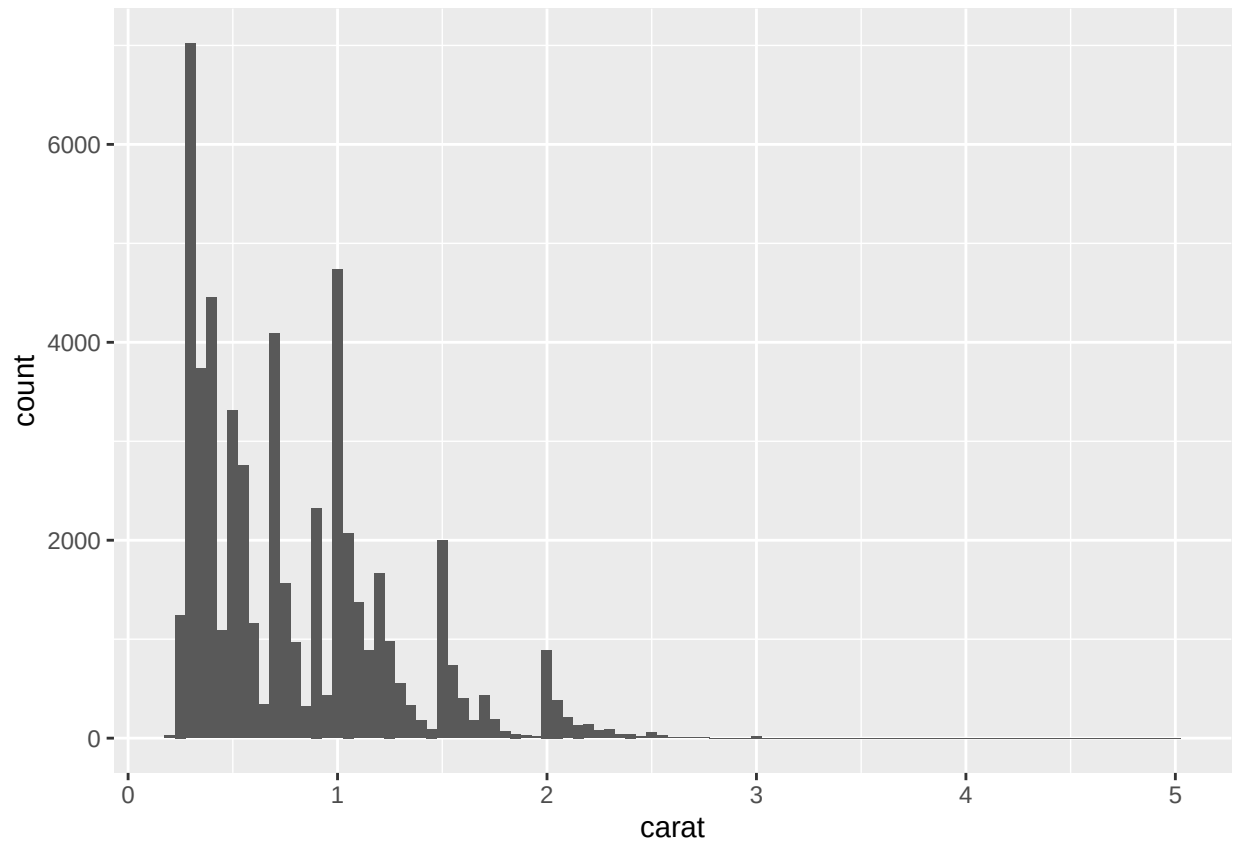
Instead of returning a value or a data frame, these functions return a plot. This works because `aes()` is a data-masking function.

Example: Imagine if you need to make a lot histograms like this:

```
diamonds |>
  ggplot(aes(x = carat)) +
  geom_histogram(binwidth = 0.1)
```

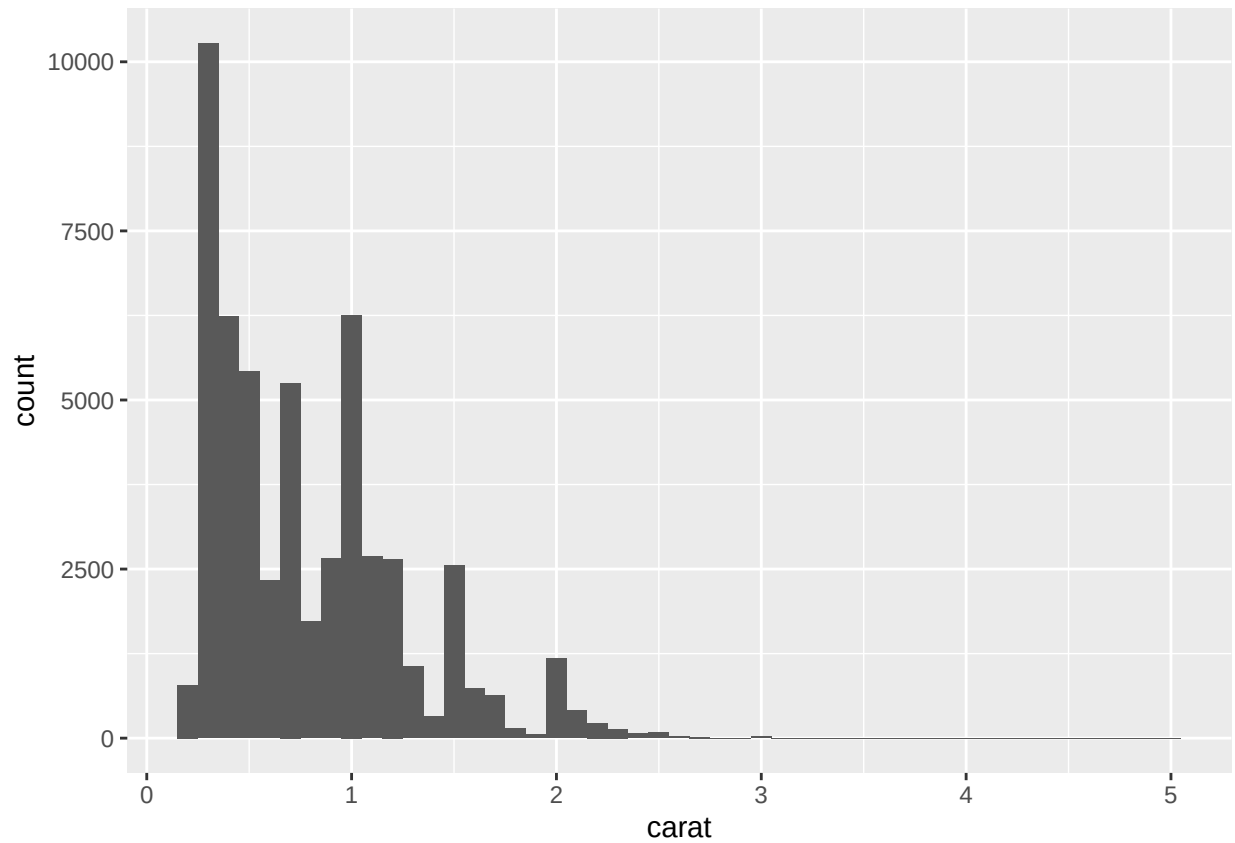


```
diamonds |>
  ggplot(aes(x = carat)) +
  geom_histogram(binwidth = 0.05)
```



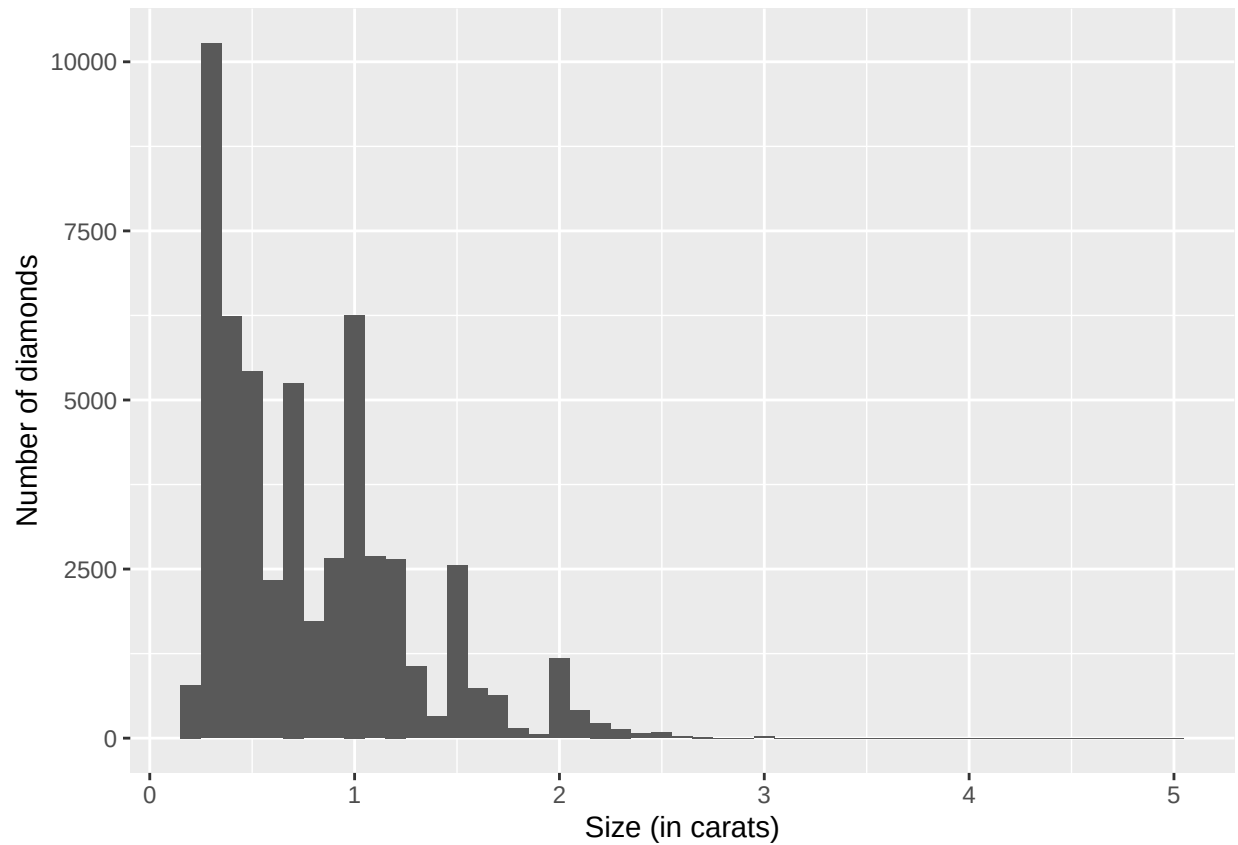
Using a function to generate the plots above (embracing is used because `aes()` is a data-masking function!):

```
histogram <- function(df, var, binwidth = NULL) {  
  df |>  
    ggplot(aes(x = {{ var }})) +  
    geom_histogram(binwidth = binwidth)  
}  
  
diamonds |> histogram(carat, 0.1)
```



The function above generates a `ggplot2` plot so additional components can be layered, just like in other plots:

```
diamonds |>
  histogram(carat, 0.1) +
  labs(
    x = "Size (in carats)",
    y = "Number of diamonds"
  )
```

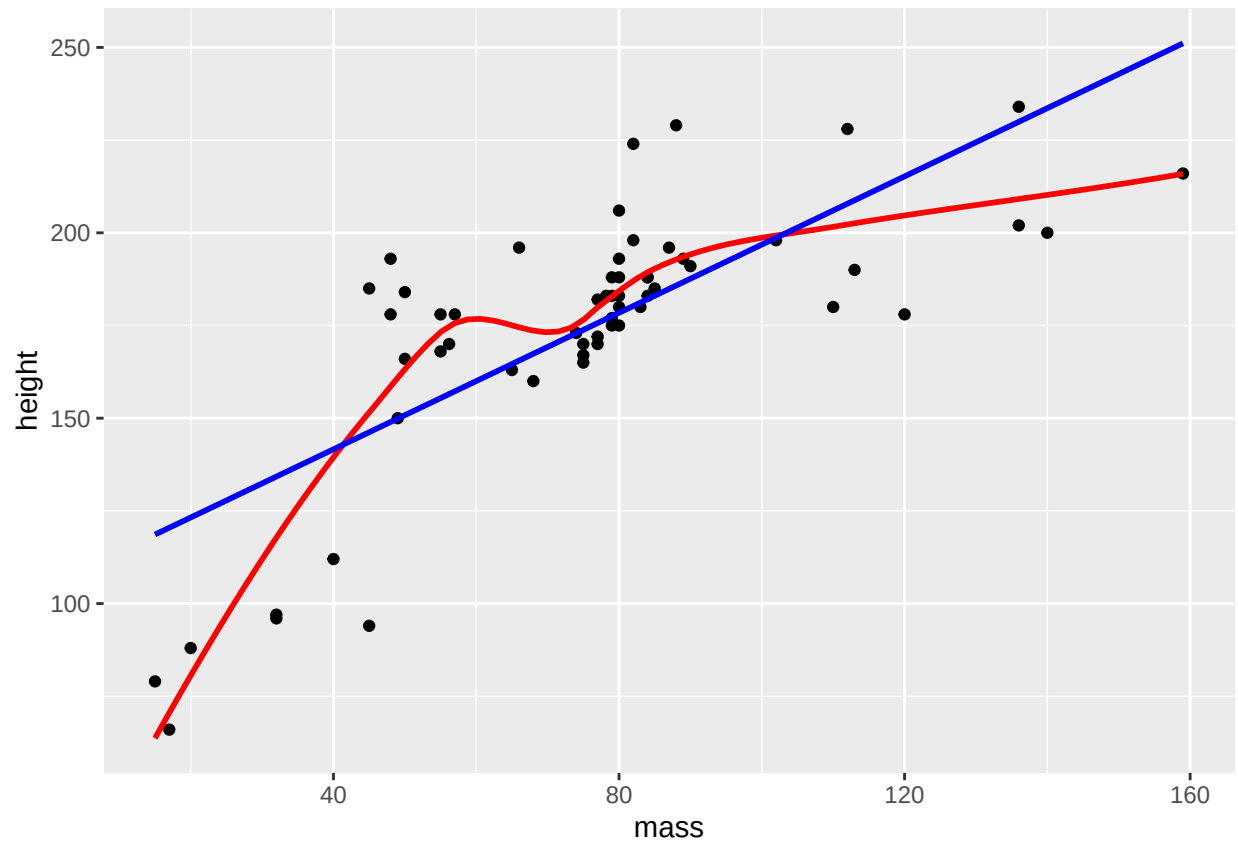


More variables

Example with overlaid plots:

```
linearity_check <- function(df, x, y) {
  df |>
    ggplot(aes(x = {{ x }}, y = {{ y }})) +
    geom_point() +
    geom_smooth(
      method = "loess",
      formula = y ~ x,
      color = "red",
      se = FALSE
    ) +
    geom_smooth(
      method = "lm",
      formula = y ~ x,
      color = "blue",
      se = FALSE
    )
}

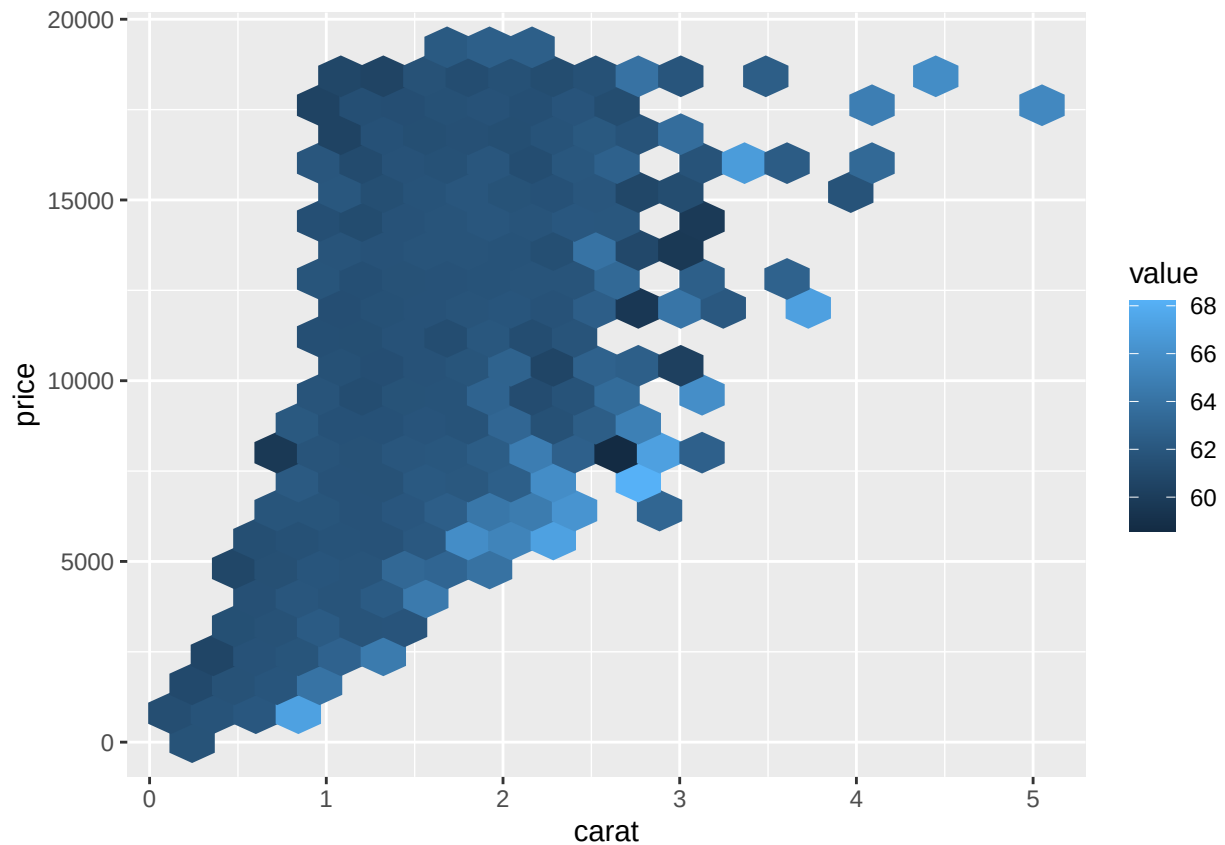
starwars |>
  filter(mass < 1000) |>
  linearity_check(mass, height)
```



Example using a function to generate a plot for a large dataset where overplotting is a problem:

```
hex_plot <- function(df, x, y, z, bins = 20, fun = "mean") {
  df |>
    ggplot(aes(x = {{ x }}, y = {{ y }}, z = {{ z }})) +
    stat_summary_hex(
      aes(color = after_scale(fill)), # make border color same as fill
      bins = bins,
      fun = fun
    )
}

diamonds |> hex_plot(carat, price, depth)
```

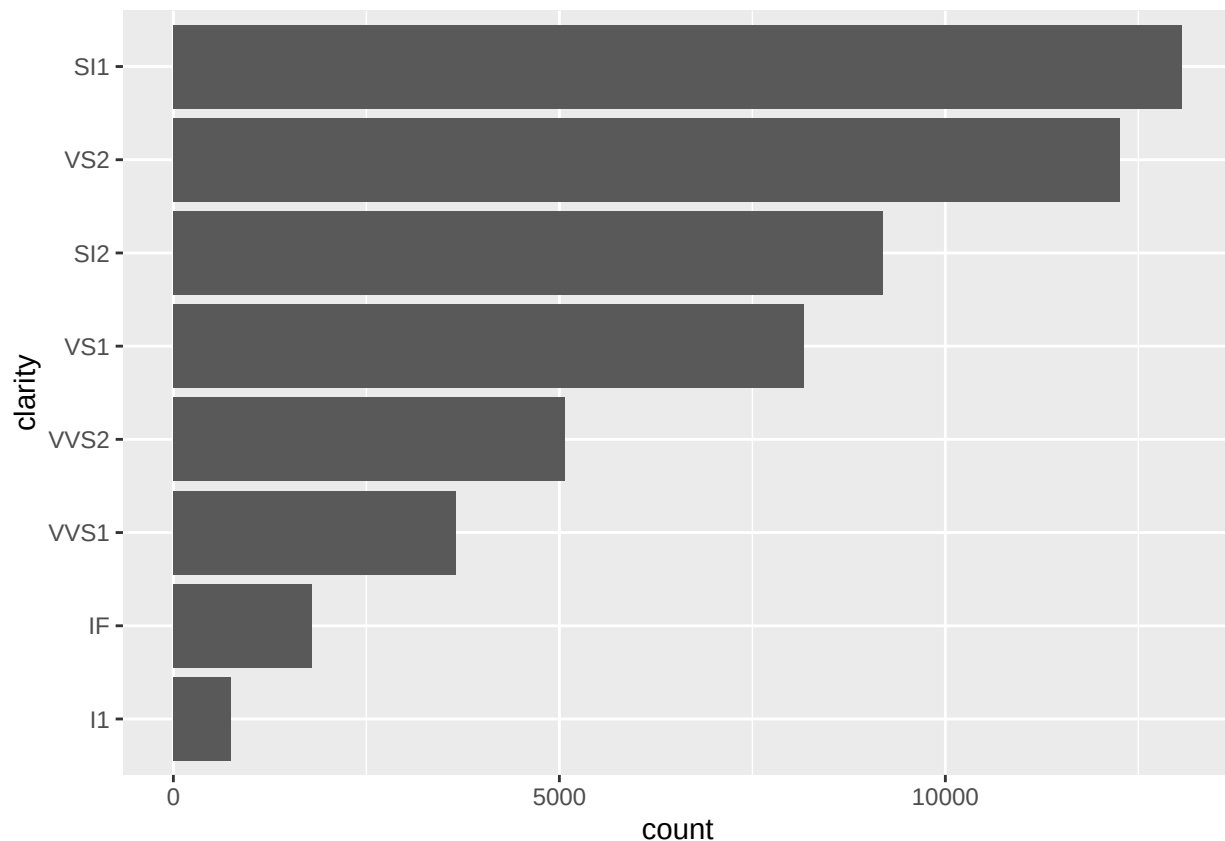


Combining with other tidyverse

A function to plot a sorted bar chart:

```
sortedBars <- function(df, var) {
  df |>
    mutate({{ var }} := fct_rev(fct_infreq({{ var }}))) |>
    ggplot(aes(y = {{ var }})) +
    geom_bar()
}

diamonds |> sortedBars(clarity)
```

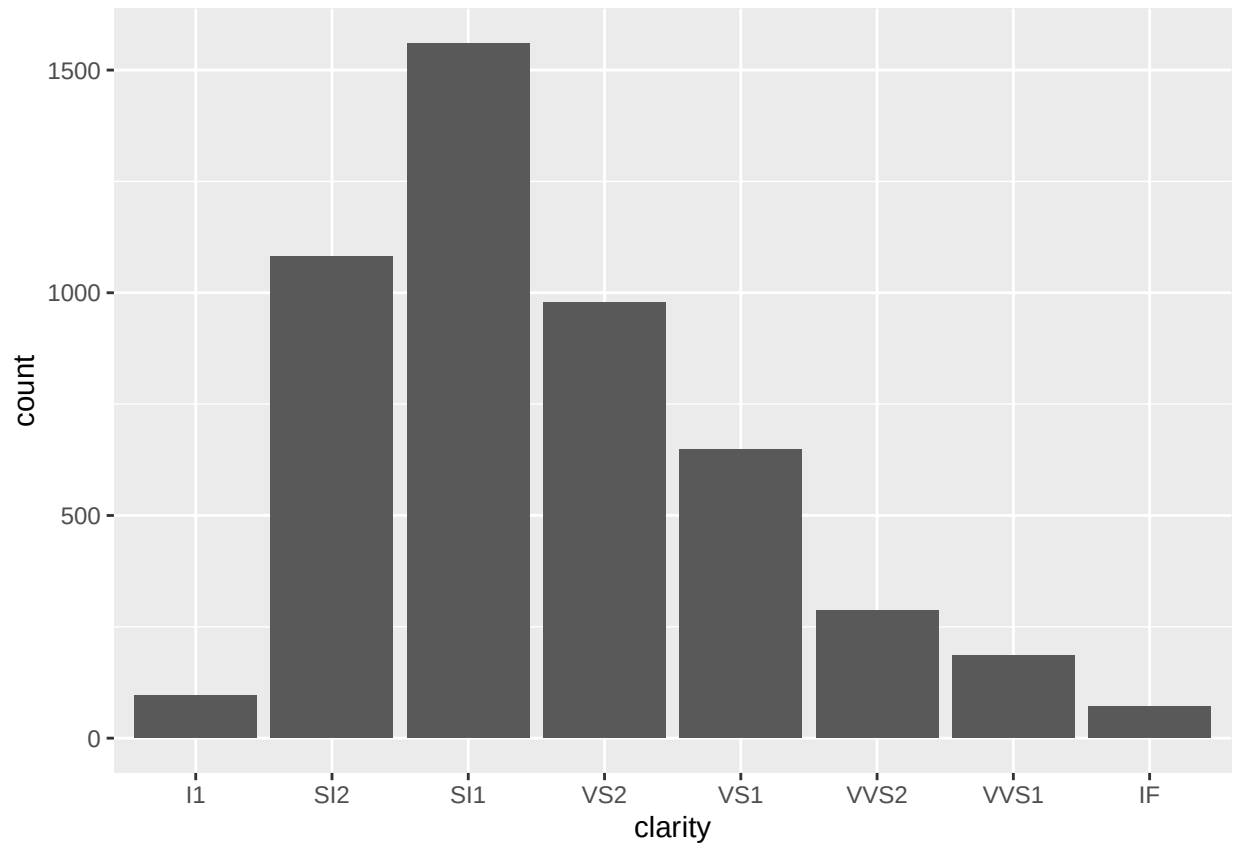


Note: the operator `:=`, commonly referred to as the “walrus operator” was used in the example to generate the variable name based on user-supplied data. This is necessary because the common behavior in R is to have only literal names on the left of the `=` operator. The walrus operator is a workaround solution to this problem.

Another example using functions to make it easier to draw a bar plot just for a subset of the data:

```
conditional_bars <- function(df, condition, var) {
  df |>
    filter({{ condition }}) |>
    ggplot(aes(x = {{ var }})) +
    geom_bar()
}

diamonds |> conditional_bars(cut == "Good", clarity)
```

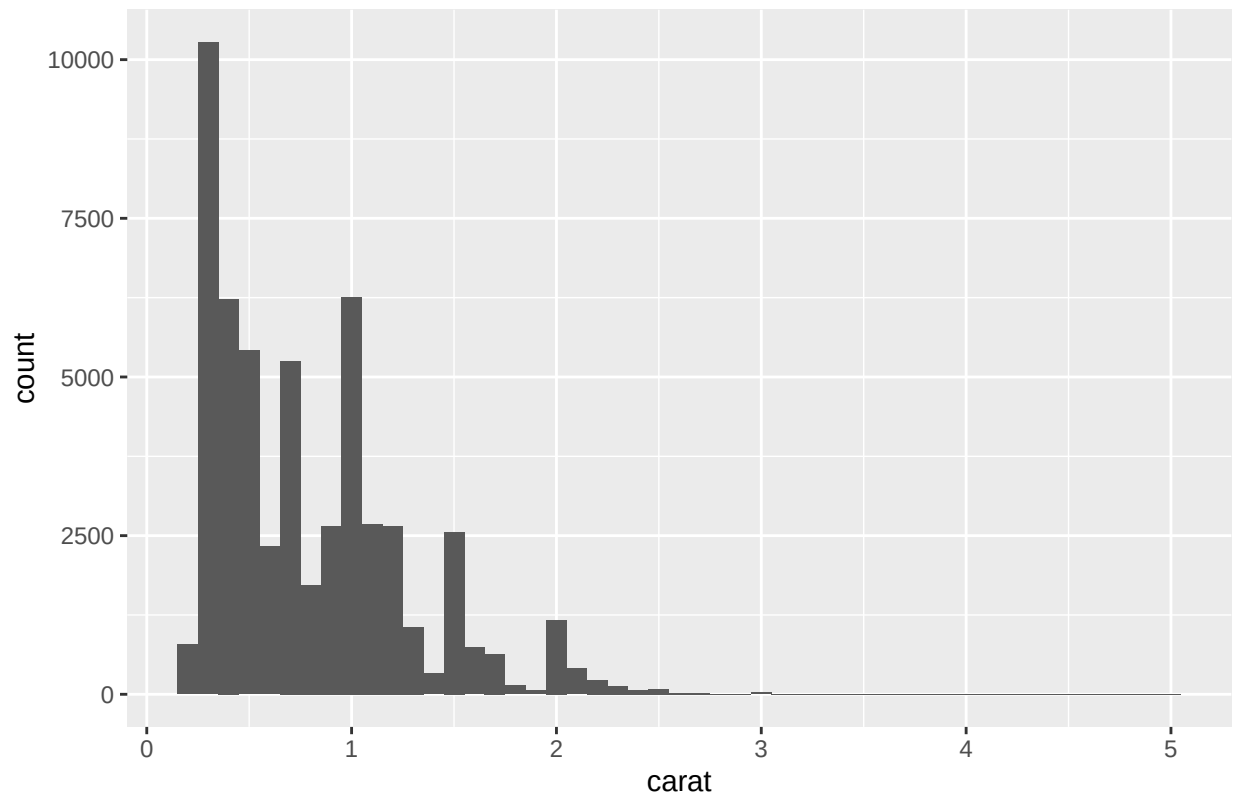


Labeling

Using `rlang::engluce()` to customize plot labels – this function works in a similar way to how Python's f-strings are used in plot labels.

```
histogram <- function(df, var, binwidth) {  
  label <- rlang::engluce("A histogram of {{var}} with binwidth {{binwidth}}")  
  df |>  
    ggplot(aes(x = {{ var }})) +  
    geom_histogram(binwidth = binwidth) +  
    labs(title = label)  
}  
  
diamonds |> histogram(carat, 0.1)
```

A histogram of carat with binwidth 0.1



Exercises

Build up a rich plotting function by incrementally implementing each of the steps below:

1. Draw a scatterplot given dataset and `x` and `y` variables.
2. Add a line of best fit (i.e. a linear model with no standard errors).
3. Add a title.

Style

General guidelines:

- The name of the function should clearly describe what the function does.
- Function names are generally verbs (except when they compute some very well-known nouns like `mean()`, or property of an object, etc.).
- Argument names are generally nouns.
- Put one space before the brackets.
- Indent the body by two spaces.
- Put an extra space before and after the variable name inside the embracing brackets (`{{}}`).